



RAG Foundations on Azure

Azure AI Search, vector stores, hybrid retrieval and the patterns that decide whether your RAG app is trustworthy — because RAG lives or dies on retrieval, not generation.

Azure AI Search

indexing

hybrid (BM25+dense)

semantic ranking

embeddings

chunking

RAG patterns

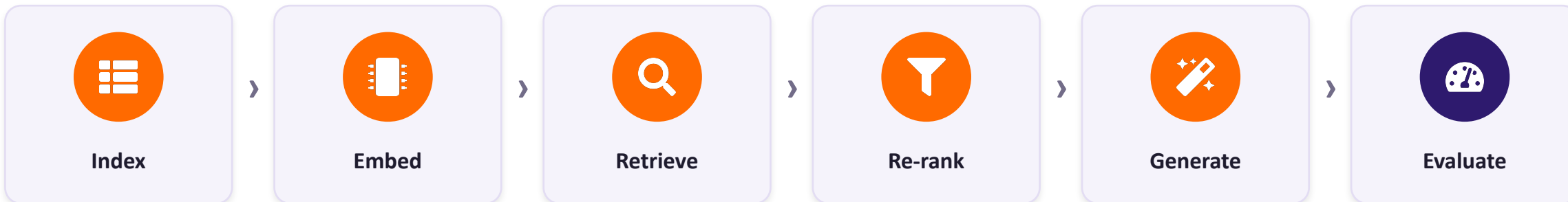
HyDE

retrieval eval

LangChain / LlamaIndex

One lens per stage — and one truth underneath them all

You built naive RAG already; it broke when context was scattered across chunks. Everything today exists to fix that. We examine each pipeline stage through **What** · **Why** · **How** · **When**. The truth underneath: **retrieval quality is RAG quality**.



The recurring measure: did the right chunk surface?

At every stage we ask one question — are the relevant chunks in the top results? That is recall@k, and it is the number the whole hour serves. Generation can only be as good as what retrieval hands it.

Stops 1–2 build the index and the vectors · Stop 3 is the patterns library · Stop 4 is the honest limits · Stop 5 measures · Stop 6 wires it together.

Six stops along the retrieval pipeline



1 • Azure AI Search

Indexing, hybrid retrieval (BM25 + dense + metadata), semantic ranking.



2 • Vector embeddings

Choosing models, chunking strategies, metadata enrichment.



3 • RAG patterns library

Naive → hybrid → re-ranker → query rewriting → HyDE.



4 • When RAG works / fails

The honest limits — and why semantic layers & KG come in Week 3.



5 • Evaluating retrieval

recall@k, MRR, nDCG, and golden test sets.



6 • LangChain & LlamaIndex

Connecting both frameworks to one Azure AI Search index.

Stops 1–2 build retrieval • 3 improves it • 4 bounds it • 5 measures it • 6 ships it.



TECHADEMY

STOP 1

Azure AI Search

Indexing · hybrid retrieval (BM25 + dense + metadata) · semantic ranking

The managed retrieval engine your RAG app stands on.



A managed search index — text, vectors and metadata in one place

Azure AI Search (formerly Azure Cognitive Search) is a managed retrieval engine. You define an **index** — a schema of fields — then query it with keyword, vector or hybrid search. It is the part of RAG that decides which chunks the model ever sees.



Integrated vectorization — the time-saver

The indexer can chunk your documents (Text Split skill) and call your embedding model automatically, at index time — no custom preprocessing code. The pipeline you hand-wrote with Chroma in Session 2 becomes a managed skillset.



Cut cost with vector compression

Scalar quantization (int8) shrinks the vector index ~4x; binary up to ~28x. Oversampling + rescoring with the original vectors recovers the relevance you'd otherwise lose — Microsoft reports nDCG held steady.

An index is just fields



Key

unique id per chunk



Content (searchable)

the chunk text — BM25



Vector

the embedding — dense search



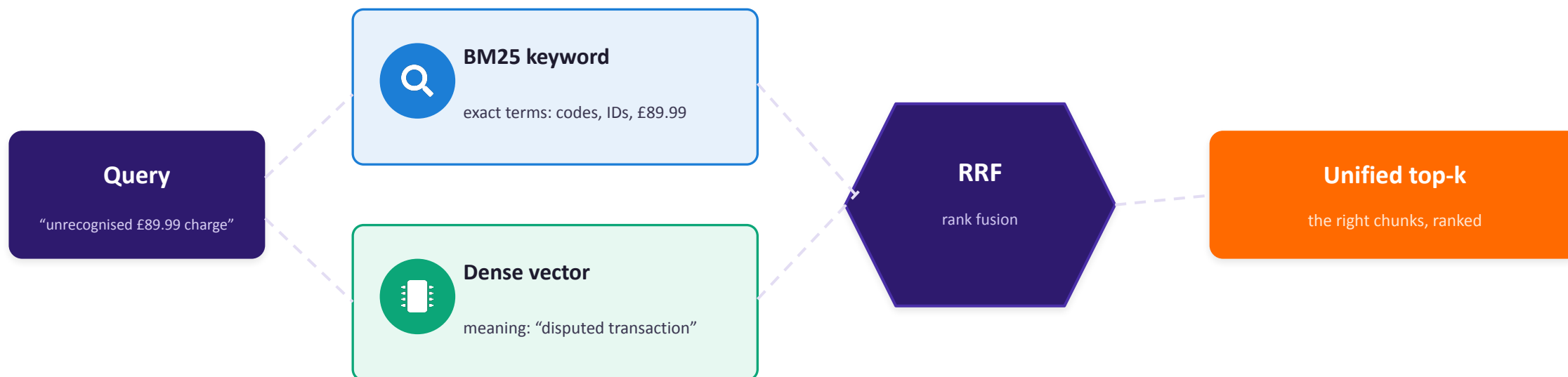
Metadata (filterable)

source, date, dept, ACL

One row = one chunk, with all three views of it.

BM25 + dense, fused by RRF — the best of both blind spots

Keyword and vector search each miss things. Run both in parallel and merge with Reciprocal Rank Fusion: chunks ranked high by either, and especially by both, rise to the top.



You already met RRF in Session 2

Reciprocal Rank Fusion sums the inverse ranks from each list, so a chunk that both searches like wins. Azure does this natively in one API call — no hand-written fusion. Set $k = 50$ so the next stage (semantic ranker) has enough candidates.

A second, smarter pass — and a way to narrow the field



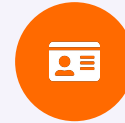
Semantic ranker (L2)

L1 BM25 / RRF returns the top ~50 candidates (fast, cheap).

L2 A transformer cross-encoder (Bing-derived) re-reads query + each candidate together and re-scores by true relevance.

Bonus returns captions and extractive answers you can show as citations. Score is @search.rerankerScore.

Microsoft's benchmarks: hybrid + semantic ranking is the strongest retrieval mode.



Metadata filtering



Narrow before you rank

“only the 2026 policy manual” beats searching the whole corpus — fewer wrong-domain chunks.



Security trimming

filter by ACL / group so a user only retrieves chunks they're allowed to see.



Freshness & routing

filter by date or department to keep answers current and on-topic.

Azure AI Search in four lines



Index = text + vector + metadata

One row per chunk, three views of it. Integrated vectorization chunks & embeds for you.



Hybrid beats either alone

BM25 for exact terms, dense for meaning, fused by RRF — native, one call.



Semantic ranker re-scores top 50

A cross-encoder L2 pass; hybrid + semantic is the strongest mode, plus captions for citations.



Filter to sharpen & secure

Metadata filters cut wrong-domain chunks and enforce who-can-see-what.

One sentence: a good index plus hybrid + semantic ranking + filters is most of your retrieval quality, before you write a line of orchestration.



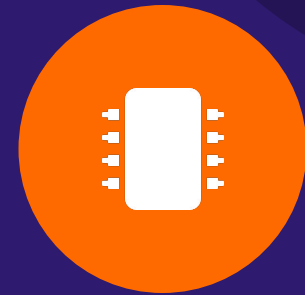
TECHADEMY

STOP 2

Vector Embeddings

Choosing models · chunking strategies · metadata enrichment

The two upstream decisions that set your ceiling.



The embedding turns text into a vector — its quality is your ceiling



WHAT

An embedding model maps text to a dense vector so similar meanings land close together. Same model for documents and queries.



WHY

Vector quality decides what's findable. If the embedding can't separate two ideas, no ranker downstream can.



HOW

Start with text-embedding-3-small (1536-d). Upgrade to 3-large (up to 3072-d) if eval shows you need it.



WHEN

Choose once, carefully: switching models means re-embedding the entire index — a full reprocessing job.

Model	Dimensions	Use when
text-embedding-3-small	1536	Default — cheap, fast, multilingual; start here
text-embedding-3-large	up to 3072 (shrinkable)	Best quality; Matryoshka lets you trim dims to save RAM
ada-002 (legacy)	1536	Existing indexes only — prefer 3-small for new work



Lock-in warning

The embedding model is the one choice you can't cheaply change — a new model means re-embedding every document and migrating the index. Run a small eval on YOUR data before you commit.

The single most impactful knob — and the one people guess at

Chunk too big and one relevant line drowns in nine irrelevant ones; too small and the context is lost. Get this wrong and nothing downstream can compensate.



Fixed / recursive

Split every N tokens (recursive respects paragraphs/sentences). Simple, fast — and often the benchmark winner.



Sentence / semantic

Split on meaning boundaries. Intuitive, but pricier and, in 2026 benchmarks, frequently no better.



Overlap (sliding window)

Repeat 10–20% between chunks so a split sentence survives — but some studies show no gain. Measure it.



Sensible default to start from

~512 tokens, 10–20% overlap. Smaller (256–512) for factoid Q&A; larger (512–1024) for multi-hop. Then tune against your eval set.



Session 2's failure was a chunking failure

Remember the answer split across chunk 1 and chunk 31? That's the boundary problem. Better chunking and overlap help — but it's also why we add hybrid, re-ranking and, in Week 3, structure.

Every chunk should carry its papers

At ingest, attach fields to each chunk. Cheap to add, and they pay off at every later stage — filtering, security, freshness and citations.



Source & citation

doc title, URL, page — so the model can cite, and users can verify.



Date & version

filter to current policy; never answer from a retired document.



Department / domain

route a query to the right sub-corpus; avoid cross-domain bleed.



ACL / security label

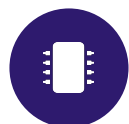
security-trim retrieval so identity decides what's even searchable.



Metadata is where retrieval meets identity

The ACL field is what lets the landing zone's identity model reach all the way into search — a user retrieves only chunks their role permits. Enrich now, or you can't filter or secure later.

The upstream decisions, four lines



Embedding sets the ceiling

Start 3-small; go 3-large if eval demands. Switching = re-embed everything, so choose once.



Chunking is the big knob

~512 tokens, 10–20% overlap as a start; recursive often beats semantic. Measure, don't guess.



Enrich every chunk

Source, date, domain, ACL — cheap at ingest, essential for filtering, security and citations.



All of it is testable

These are hyperparameters. Stop 5 shows how to tune them against a golden set.

One sentence: hybrid search can only rank what good embeddings and good chunks made findable — the ceiling is set here.



TECHADEMY

STOP 3

RAG Patterns Library

Naive → hybrid → re-ranker → query rewriting → HyDE

A toolbox, from the baseline you built to the clever moves.



Three places to improve retrieval — and the baseline they fix



Pre-retrieval

Reshape the QUERY before searching: query rewriting, HyDE, sub-questions.



Retrieval

Improve the SEARCH itself: hybrid (BM25 + dense), metadata filters.



Post-retrieval

Reorder what came back: re-ranking, then optimise the prompt.



Naive RAG — the baseline you built in Session 2

Embed the query → top-k vector search → stuff chunks into the prompt. It works on easy, well-phrased questions over a clean corpus. It fails on exact terms (no keyword path), scattered context (chunk 1 vs chunk 31), ambiguous wording, and anything needing more than one hop. Every pattern below is a targeted fix for one of those failures — you don't need all of them at once.

Five moves — what each is, what it fixes, what it costs

Pattern	What it does	Fixes	Cost
Naive	Embed query → top-k vector → prompt	Nothing extra — the baseline	Lowest
Hybrid	Add BM25 + RRF alongside vector	Exact terms, codes, names	~none (native)
Re-ranker	Cross-encoder re-scores top ~50	Wrong order; near-misses ranked low	+50–200 ms
Query rewriting	LLM rewrites query into variants / sub-questions	Ambiguous & broad queries; recall	+1 LLM call
HyDE	LLM drafts a hypothetical answer; embed THAT	Query–document vocabulary mismatch	+1 LLM call



Stack them in order of cheapness

Hybrid first (free), then a re-ranker (cheap latency), then pre-retrieval tricks (query rewriting / HyDE, an extra LLM call) only where eval shows recall is still short. Don't pay for what your golden set says you don't need.

Reshape the question so it looks like the answer



HyDE — hypothetical doc embeddings



1 · The LLM drafts a fake but plausible answer to the question.



2 · Embed that hypothetical answer — not the raw query.



3 · Search with it: answers look like answers, so the vector lands nearer real documents.



Why: a short question and a full document live in different parts of vector space. HyDE closes that gap — strong for zero-shot recall.



Query rewriting & decomposition



Multi-query

rewrite the question several ways, retrieve for each, fuse the results — more recall.



Sub-questions

decompose a complex question into parts, answer each, combine (multi-hop).



Routing

send the query to the right index / tool by its metadata.

Built into LlamaIndex (HyDEQueryTransform, SubQuestionQueryEngine) and LangChain (MultiQueryRetriever).



TECHADEMY

STOP 4

When RAG Works — and Fails

The honest limits · and why semantic layers & KG wait for Week 3

Know the edge of the map before you fall off it.



RAG is brilliant at some questions and structurally bad at others



RAG works well when...



The answer sits in one or a few passages you can retrieve.



It's factual lookup over your own / fresh / proprietary text.



You need grounding and citations to cut hallucination.



Questions are reasonably well-phrased and on-topic.



RAG fails when...



Context is scattered across many chunks (chunk 1 + chunk 31).



The question is multi-hop or needs synthesis across docs.



It's relational — 'who reports to whom', 'which clauses conflict'.



It's global — 'summarise the whole corpus' — or not in the corpus at all.

When retrieval tricks stop helping — you need structure

Hybrid, re-ranking, HyDE and better chunking all push recall higher. But they all share one limit: a pile of chunks has no model of how facts relate. Some questions need that model.



Knowledge graphs

Entities and relationships made explicit — answer relational & multi-hop questions a chunk pile can't.



Semantic layers

A governed model of your business terms over raw data — consistent meaning, not just text matches.



Hierarchy / structure-aware

Walk a document's structure (e.g. page-index) with LLM reasoning — powerful, but costlier per query.



That's Week 3 — semantic layers & knowledge graphs

Today's job is to make vector + hybrid retrieval as good as it can be, and to KNOW its ceiling. When recall plateaus on relational or multi-hop questions, the answer isn't a bigger model — it's structure. We build that next week.



TECHADEMY

STOP 5

Evaluating Retrieval

recall@k · MRR · nDCG · golden test sets

You cannot improve what you refuse to measure.



A golden test set, then the right metric for the question



Start with a golden test set

A curated list of real queries paired with the chunk(s) that truly answer them — your ground truth. 50–100 honest questions from real users beats a thousand demo questions. This is your retrieval contract test, the Session-4 discipline applied to search.

Metric	Answers the question...	Reach for it when...
recall@k	Did the right chunk make the top k at all?	Always — the first metric for RAG
precision@k	How much of the top k is actually relevant?	Top-k is noisy / prompt is polluted
MRR	How high was the FIRST relevant result?	One right answer; rank matters
nDCG	Is the whole ranking good, by graded relevance?	Many relevant docs; order matters

Microsoft uses nDCG to prove vector compression keeps relevance — the same metric you'll use to defend your own changes.

If the chunk isn't retrieved, no model can save you



recall@k first

Low recall@k is a hard ceiling. A perfect prompt and the best GPT can't answer from a chunk that never entered the context. Fix retrieval before you touch the prompt.

Diagnose: low recall → fix chunking / embeddings / hybrid. Good recall, bad answers → fix the prompt.

The tuning loop



Build the golden set

real queries → known-good chunks



Measure recall@k / nDCG

the honest baseline



Change ONE thing

chunk size, hybrid on, add re-ranker



Re-measure & keep wins

regression-test every change

Tools: Azure AI evaluation · TruLens · Ragas — wire eval into CI.



TECHADEMY

STOP 6

LangChain & LlamaIndex on Azure



Connecting both frameworks to Azure AI Search

Two frameworks, one index — wire it together.

Both speak to the same Azure AI Search index — pick by job



LlamaIndex



Retrieval-centric — the deepest built-in advanced retrieval.



HyDEQueryTransform, SubQuestionQueryEngine, fusion retriever out of the box.



Reach for it when retrieval IS the app: Q&A, search, knowledge.



LangChain



Orchestration-centric — chains, tools, agents, broad ecosystem.



AzureSearch vector store, AzureAISearchRetriever, MultiQueryRetriever.

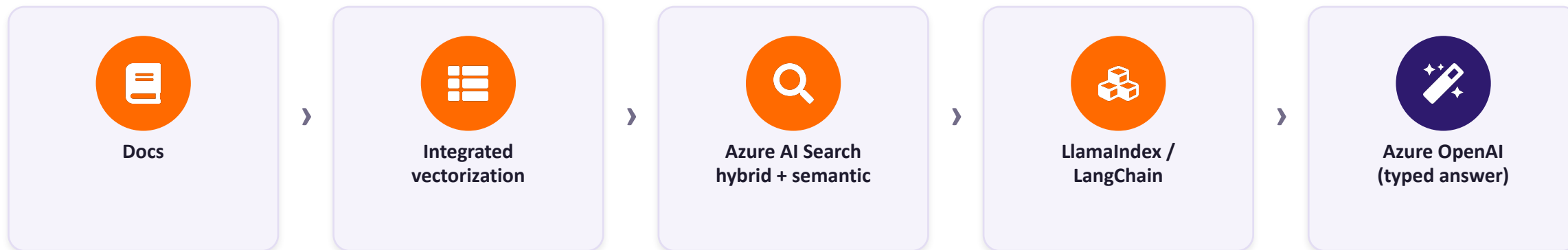


Reach for it when retrieval is one tool inside a bigger agent flow.

Same index, either framework — keyless via the identity from last session

```
# LlamaIndex from llama_index.vector_stores.azureaisearch import AzureAISearchVectorStore
# LangChain  from langchain_community.vectorstores.azuresearch import AzureSearch
```

The whole pipeline, on Azure, in the landing zone you already have



It sits inside last session's foundation

Same EU/UK-resident region. Same managed identity — keyless to Search and OpenAI. Same RBAC and private endpoints. Metadata ACLs security-trim retrieval so identity reaches into the index. RAG didn't escape the landing zone; it lives in it.



And it closes the loop with eval

Wrap the answer in your Week-1 Pydantic type, and gate the pipeline with a golden-set recall@k check in CI. Typed output + measured retrieval = a RAG system you can defend to a regulator, not just demo.

The hour in one checklist — and one truth



I index text + vectors + metadata, and let integrated vectorization chunk & embed



I add patterns (hybrid → re-ranker → query rewriting / HyDE) cheapest-first



I retrieve hybrid (BM25 + dense + RRF) and re-rank the top 50 semantically



I know RAG's ceiling — relational & multi-hop wait for Week 3 structure



I choose the embedding once — switching means re-embedding everything



I build a golden set and lead with recall@k before touching the prompt



I treat chunking as the big knob and tune it against an eval set



I pick LlamaIndex or LangChain over the same index, keyless, in the landing zone

The truth through all eight: retrieval quality is RAG quality. Fix what the model sees before you change how it speaks.



The one idea to keep

A better model can't answer from a chunk it never saw.



Index well, retrieve hybrid, re-rank, and measure recall@k — then RAG becomes a system you can trust, not a demo you hope holds. Next week: when retrieval hits its ceiling, we add structure — semantic layers & knowledge graphs.

Questions — and bring a query your current RAG gets wrong; we'll diagnose it live.